

Latent Variable Models (Logistic Regression)

Foundations of Machine Learning

Q1. Please answer the following questions in your own words:

1. Why do we use general linear models/latent variable models/activation functions in analysis? What advantage do they have over linear models?
2. Explain why binary and categorical cross entropy are effective loss functions for fitting logistic regression models.
3. True or false, and explain: Logistic regression is a linear model.
4. True or false, and explain: Logistic regression cannot be used for classification.
5. Does the coefficient in a logistic regression represent the change in the predicted value for a 1 unit change in the feature/covariate/explanatory variable?
6. True or false, and explain: Feature engineering is unnecessary for logistic regression, because it is a nonlinear model.
7. True or false, and explain: Logistic regression is an improvement over the usual linear model (ordinary least squares), and you should always use logistic regression instead.

#1

General linear models allow us to study complex data that does not follow a simple straight line, such as yes or no outcomes. Latent variable models help us find hidden patterns of invisible factors that we cannot measure directly but that connect all our data points together. Activation functions are essential because they allow models to learn complicated relationships, which is a massive advantage over standard linear models that can only understand simple patterns.

#2

These loss functions are effective because they heavily punish the model with a high penalty score when it is confidently wrong about a category. Instead of just checking if the guess is right, they measure how close the predicted probability is to the actual answer, which helps the model fine-tune its math more precisely. This approach works better than simple errors

because it forces the model to become very sure of the correct labels rather than just guessing near the middle.

#3

True, logistic regression is considered a linear model because it calculates a weighted sum of your inputs to create what I call a "decision boundary", which is a straight line or flat plane. While the final output is a probability between zero and one, the underlying math that determines the result relies entire on this linear combination of the data.

#4

False, because while regression is in the name, it is primarily used as a classification tool to predict which category an item belongs to. It works by taking the model's numerical output and turning it into a probability, which allows it to sort data into specific groups like yes or no.

#5

No, the coefficient does not show a simple plus or minus change in the final answer, because the relationship is a curve rather than a straight line. Instead, it tells you how much the strength of the evidence for an outcome increases or decreases for everyone one unit change in your variable.

#6

False, because logistic regression is actually a linear model that needs you to manually create shape in your data to catch complex patterns. If your data has curvy relationships, you must use feature engineering to create new variables so the model can see those patterns clearly.

#7

False, because both models are simply tools designed for different types of jobs. You use ordinary least squares to

predict specific numbers like price or height, while you use logistic regression only when you need to sort things into categories like yes or no.

Q2. This is a question about student dropout probability.

The relevant data are in `data.csv` and the file is semicolon-separated.

The relevant variables are:

- `Debtor` - `1` if the student has outstanding debt and `0` otherwise
- `Tuition fees up to date` - `1` if tuition is up to date and `0` otherwise
- `Scholarship holder` - `1` if the student has a scholarship and `0` otherwise
- `Age at enrollment` - student's age when entering the program
- `Curricular units 1st sem (approved)` - number of first-semester courses passed
- `Target` - `Dropout`, `Graduate`, or `Enrolled`

1. Load the data, `data.csv`. Be sure to read it with the correct delimiter. Filter on columns to the variables we'll actually use. Document how you handle any missing values. For parts 2-5, create a binary target where `Dropout` corresponds to dropout and `Graduate` or `Enrolled` corresponds to not dropout.
2. Run a logistic regression of dropout status on `Debtor`, `Tuition fees up to date`, `Scholarship holder`, and `Age at enrollment`. Which variables predict a higher dropout probability, and which predict a lower dropout probability? Does being up to date on tuition seem to reduce dropout risk?
3. Use `.predict_proba` and compute expected dropout probabilities. Make a scatter plot of your predicted dropout probability against `Age at enrollment`, hued by `Tuition fees up to date`. For what ages does being up to date on tuition seem to reduce dropout probability the most? On average, by how much does being up to date on tuition change the predicted probability of dropout?
4. Use `.predict` and compute a confusion matrix for your classifier. What is the accuracy?
5. Compute predicted dropout probabilities with a linear model with the same variables, and predict dropout probabilities. Make a histogram of predicted probabilities for both logistic regression and the linear model. Compare their performance.
6. Based on your above work, which students are the most at-risk of dropping out? What kinds of interventions can you imagine that would help at-risk students stay enrolled?
7. Run a multinomial logistic regression of `Target` on `Debtor`, `Tuition fees up to date`, and `Curricular units 1st sem (approved)`. Use `.predict` to make a hard classification, and compute a confusion matrix. Describe your results. Use `.predict_proba` to predict probabilities instead and look at the values. Does your

hard classification predict every class? Do your predicted probabilities predict every class?

#1

```
In [22]: import pandas as pd

df = pd.read_csv('data.csv', sep=';')

relevant_cols = [
    'Debtor',
    'Tuition fees up to date',
    'Scholarship holder',
    'Age at enrollment',
    'Curricular units 1st sem (approved)',
    'Target'
]
df = df[relevant_cols]

df = df.dropna()

df['Target'] = df['Target'].apply(lambda x: 1 if x == 'Dropout' else 0)
```

#2

```
In [23]: import statsmodels.api as sm

features = ['Debtor', 'Tuition fees up to date', 'Scholarship holder', 'Age at enro

X = sm.add_constant(df[features])
y = df['Target'] # already 0/1 from Part 1

model = sm.Logit(y, X)
result = model.fit()

print(result.summary())
```

Optimization terminated successfully.
 Current function value: 0.500966
 Iterations 6

Logit Regression Results

```

=====
Dep. Variable:          Target    No. Observations:          4424
Model:                  Logit     Df Residuals:              4419
Method:                  MLE      Df Model:                  4
Date:                   Wed, 08 Apr 2026    Pseudo R-squ.:            0.2020
Time:                   02:30:20    Log-Likelihood:            -2216.3
converged:              True      LL-Null:                  -2777.3
Covariance Type:        nonrobust    LLR p-value:              1.298e-241
=====
=====
                                coef    std err          z      P>|z|      [0.025
0.975]
-----
const                0.5352      0.182      2.943      0.003      0.179
0.892
Debtor                0.5272      0.125      4.206      0.000      0.282
0.773
Tuition fees up to date -2.6056      0.142     -18.331      0.000     -2.884
-2.327
Scholarship holder    -1.2423      0.107     -11.573      0.000     -1.453
-1.032
Age at enrollment      0.0498      0.005      10.624      0.000      0.041
0.059
=====
=====

```

Which variables predict a higher dropout probability, and which predict a lower dropout probability? Does being up to date on tuition seem to reduce dropout risk?

Higher dropout probability: Being a debtor and older age at enrollment both increase dropout risk.

Lower dropout probability: Being up to date on tuition and holding a scholarship both strongly reduce dropout risk, yes staying current on tuition is the stronger predictor of not dropping out.

#3

```

In [24]: predicted_probs = result.predict(X)
df['dropout_prob'] = predicted_probs

print(df['dropout_prob'].describe())

```

```

count    4424.000000
mean      0.321203
std       0.232478
min       0.078327
25%      0.236279
50%      0.254737
75%      0.360067
max       0.989558
Name: dropout_prob, dtype: float64

```

```

In [25]: import matplotlib.pyplot as plt

colors = df['Tuition fees up to date'].map({1: 'steelblue', 0: 'tomato'})

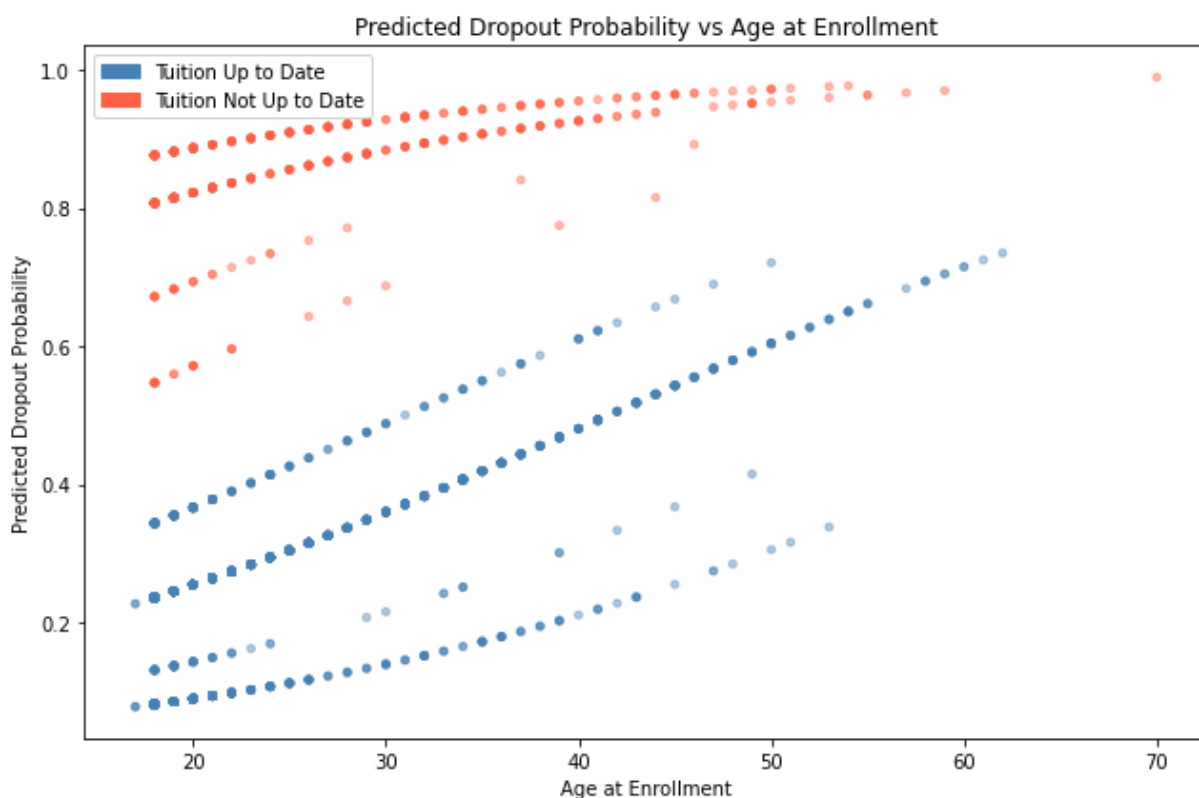
plt.figure(figsize=(9, 6))
scatter = plt.scatter(df['Age at enrollment'], df['dropout_prob'], c=colors, alpha=

import matplotlib.patches as mpatches
legend_handles = [mpatches.Patch(color='steelblue', label='Tuition Up to Date'), mp

plt.legend(handles=legend_handles)

plt.xlabel('Age at Enrollment')
plt.ylabel('Predicted Dropout Probability')
plt.title('Predicted Dropout Probability vs Age at Enrollment')
plt.tight_layout()
plt.show()

```



```

In [26]: mean_not_uptodate = df[df['Tuition fees up to date'] == 0]['dropout_prob'].mean()
mean_uptodate = df[df['Tuition fees up to date'] == 1]['dropout_prob'].mean()
print(mean_not_uptodate - mean_uptodate)

```

0.6180970381432417

For what ages does being up to date on tuition seem to reduce dropout probability the most? On average, by how much does being up to date on tuition change the predicted probability of dropout?

Being up to date on tuition reduces dropout probability the most at older ages (50-70), where the gap between red and blue dots is widest. On average, being up to date on tuition reduces predicted dropout probability by about 0.618, making it by far the most impact variable in the model.

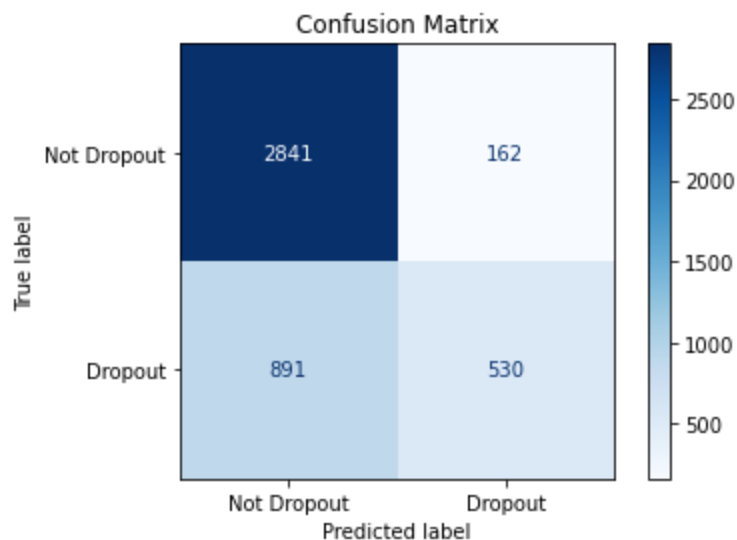
#4

```
In [27]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

df['predicted'] = (df['dropout_prob'] >= 0.5).astype(int)

cm = confusion_matrix(df['Target'], df['predicted'])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Not Dropout', 'Dropout'], cmap='Blues')
disp.plot(cmap='Blues')
plt.title('Confusion Matrix')
plt.show()

# print accuracy
accuracy = (df['predicted'] == df['Target']).mean()
print(f"Accuracy: {accuracy:.4f}")
```



Accuracy: 0.7620

#5

```
In [29]: import statsmodels.api as sm

model_lpm = sm.OLS(y, X)
```

```

result_lpm = model_lpm.fit()

print(result_lpm.summary())

df['dropout_prob_lpm'] = result_lpm.predict(X)

print(df['dropout_prob_lpm'].describe())

```

OLS Regression Results

```

=====
Dep. Variable:          Target    R-squared:                0.245
Model:                  OLS       Adj. R-squared:           0.244
Method:                 Least Squares   F-statistic:             358.8
Date:                  Wed, 08 Apr 2026   Prob (F-statistic):      7.49e-268
Time:                  02:42:51    Log-Likelihood:         -2286.2
No. Observations:      4424        AIC:                    4582.
Df Residuals:          4419        BIC:                    4614.
Df Model:              4
Covariance Type:       nonrobust
=====

```

```

=====
coef      std err          t      P>|t|      [0.025
0.975]
-----
const      0.5857      0.030     19.443     0.000     0.527
0.645
Debtor      0.0852      0.021      4.043     0.000     0.044
0.126
Tuition fees up to date -0.5122      0.021    -24.436     0.000    -0.553
-0.471
Scholarship holder -0.1765      0.014    -12.196     0.000    -0.205
-0.148
Age at enrollment   0.0095      0.001     11.433     0.000     0.008
0.011
=====
Omnibus:      459.825   Durbin-Watson:           1.966
Prob(Omnibus): 0.000   Jarque-Bera (JB):        571.631
Skew:          0.858   Prob(JB):                7.45e-125
Kurtosis:      2.606   Cond. No.                 143.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

count    4424.000000
mean      0.321203
std       0.231217
min       0.058320
25%       0.244279
50%       0.263253
75%       0.348636
max       1.334954
Name: dropout_prob_lpm, dtype: float64

```

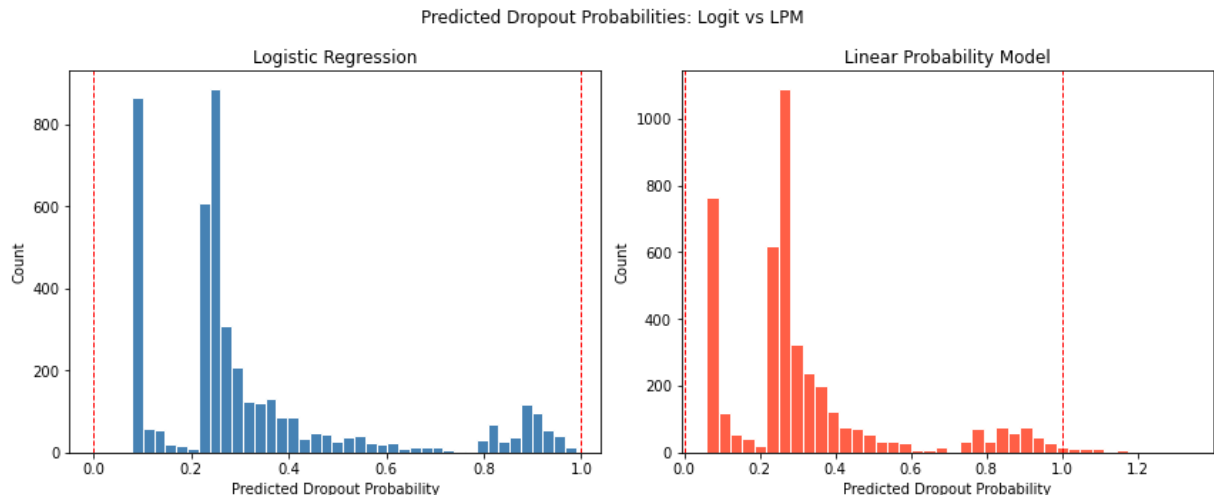


```
In [31]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].hist(df['dropout_prob'], bins=40, color='steelblue', edgecolor='white')
axes[0].set_title('Logistic Regression')
axes[0].set_xlabel('Predicted Dropout Probability')
axes[0].set_ylabel('Count')
axes[0].axvline(0, color='red', linestyle='--', linewidth=1)
axes[0].axvline(1, color='red', linestyle='--', linewidth=1)

axes[1].hist(df['dropout_prob_lpm'], bins=40, color='tomato', edgecolor='white')
axes[1].set_title('Linear Probability Model')
axes[1].set_xlabel('Predicted Dropout Probability')
axes[1].set_ylabel('Count')
axes[1].axvline(0, color='red', linestyle='--', linewidth=1)
axes[1].axvline(1, color='red', linestyle='--', linewidth=1)

plt.suptitle('Predicted Dropout Probabilities: Logit vs LPM')
plt.tight_layout()
plt.show()
```



#6

Which students are the most at-risk of dropping out? What kinds of interventions can you imagine that would help at-risk students stay enrolled?

Most at risk students: Students who are behind on tuition, have outstanding debt, lack a scholarship, and enrolled at an older age face the highest predicted dropout probability. The model shows tuition status alone shifts dropout risk by over 60 percentage points.

Interventions: Universities could target these students with emergency financial aid or flexible tuition payment plans to address the tuition and debt factors, and pair that with dedicated advising outreach for older enrollees who may be juggling work or family responsibilities alongside school.

#7

```
In [36]: from sklearn.linear_model import LogisticRegression

df_original = pd.read_csv('data.csv', sep=';')
df_original = df_original[relevant_cols].dropna()

features_multi = ['Debtor', 'Tuition fees up to date', 'Curricular units 1st sem (a

X_multi = df[features_multi]
y_multi = df_original['Target']

model_multi = LogisticRegression(multi_class='multinomial', solver='lbfgs', max_ite
model_multi.fit(X_multi, y_multi)

print("Classes:", model_multi.classes_)
print(" ")
print("Coefficients:")
print(pd.DataFrame(model_multi.coef_, index=model_multi.classes_, columns=features_
```

Classes: ['Dropout' 'Enrolled' 'Graduate']

Coefficients:

	Debtor	Tuition fees up to date \
Dropout	0.367707	-1.746371
Enrolled	0.272727	0.223398
Graduate	-0.640434	1.522973

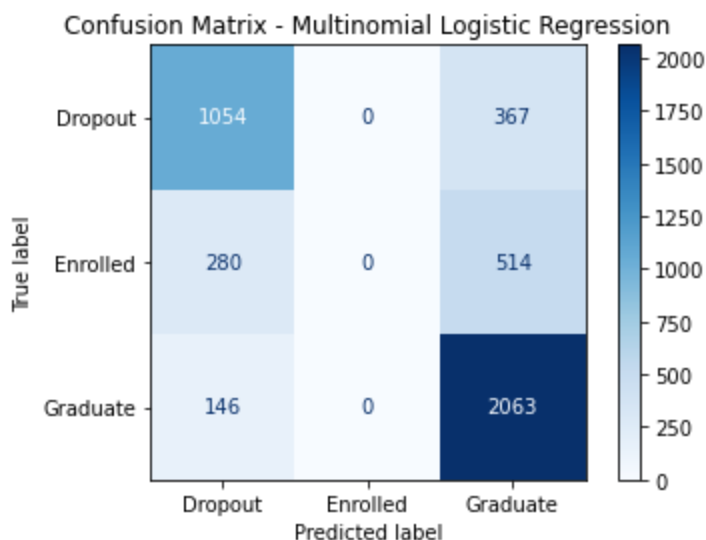
	Curricular units 1st sem (approved)
Dropout	-0.288861
Enrolled	-0.025555
Graduate	0.314416

```
In [37]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

y_pred_multi = model_multi.predict(X_multi)

cm_multi = confusion_matrix(y_multi, y_pred_multi, labels=model_multi.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_multi, display_labels=model_multi
disp.plot(cmap='Blues')
plt.title('Confusion Matrix - Multinomial Logistic Regression')
plt.show()

accuracy = (y_pred_multi == y_multi).mean()
print(f"Accuracy: {accuracy:.4f}")
```



Accuracy: 0.7046

Describe your results from above

The model fails to predict the enrolled class, classifying every student as either a Dropout or a Graduate, most likely because Enrolled is a minority class that gets overwhelmed by the other two. Overall accuracy is 70.5%, driven largely by strong performance on Graduates and a decent performance on Dropouts, but the model is essentially a binary classifier in practice.

```
In [38]: proba_multi = model_multi.predict_proba(X_multi)

df_proba = pd.DataFrame(proba_multi, columns=model_multi.classes_)
print(df_proba.describe())
```

	Dropout	Enrolled	Graduate
count	4.424000e+03	4424.000000	4424.000000
mean	3.212071e-01	0.179473	0.499320
std	2.866758e-01	0.066331	0.287546
min	9.612430e-07	0.000280	0.002137
25%	1.178304e-01	0.140654	0.172857
50%	1.842278e-01	0.176979	0.603121
75%	5.897093e-01	0.212651	0.705190
max	9.601763e-01	0.327905	0.999719

Does your hard classification predict every class? Do your predicted probabilities predict every class?

The hard classification does not predict every class, it never predicted enrolled, assigning every student to either Dropout or Graduate. However, the predicted probabilities do cover all three class, with Enrolled probabilities ranging from 0.00028 to 0.33, meaning the model sees some chance of Enrolled for every student but never high enough to win the hard classification vote.

Q3. This is a question about 10-year survival rate for cirrhosis of the liver.

The relevant data are:

- **Bilirubin** - A liver byproduct produced when red blood cells break down
- **Edema** - Swelling in limbs; **N** for none, **S** for edema resolvable with diuretics, **Y** for edema despite diuretics
- **Drug** - D-penicillamine, or placebo
- **Stage** - current state of disease, from 1 to 4
- **Status** - **C** or **CL** corresponds to alive, and **D** corresponds to dead

1. Load the data, `cirrhosis.csv`. Filter on columns to the variables we'll actually use. Document how you handle any missing values.
2. Run a logistic regression of **Status** on **Edema**, **Drug**, and **Bilirubin**. Does the drug improve survival probability? Does higher or lower bilirubin predict a higher or lower survival rate? How does survival rate vary with edema?
3. Use `.predict_proba` and compute expected survival probabilities. Make a scatter plot of your predicted survival probability against **Bilirubin**, hued by **Drug**. For what values of **Bilirubin** does the drug seem to increase patient survival rate? On average, by how much do patients who take the drug increase their probability of survival?
4. Use `.predict` and compute a confusion matrix for your classifier. What is the accuracy?
5. Compute predicted survival probabilities with a linear model with the same variables, and predict survival probabilities. Make a histogram of predictive probabilities for both logistic regression and the linear model. Compare their performance.
6. Run a logistic regression of **Stage** on **Edema** and **Bilirubin**. Use `.predict` to make a hard classification, and compute a confusion matrix. Describe your results. Use `.predict_proba` to predict probabilities instead and look at the values. Does your hard classification predict every class? Do your predicted probabilities predict every class?

#1

```
In [42]: import pandas as pd

df_c = pd.read_csv('cirrhosis.csv')

relevant_cols_c = ['Bilirubin', 'Edema', 'Drug', 'Stage', 'Status']
df_c = df_c[relevant_cols_c]

print(df_c.shape)
print(df_c.head())
```

```
print(f"\nMissing values:\n{df_c.isnull().sum()}")
print(f"\nStatus value counts:\n{df_c['Status'].value_counts()}")
print(f"\nEdema value counts:\n{df_c['Edema'].value_counts()}")

df_c = df_c.dropna()
print(" ")
print(df_c.shape)
```

(418, 5)

	Bilirubin	Edema	Drug	Stage	Status
0	14.5	Y	D-penicillamine	4.0	D
1	1.1	N	D-penicillamine	3.0	C
2	1.4	S	D-penicillamine	4.0	D
3	1.8	S	D-penicillamine	4.0	D
4	3.4	N	Placebo	3.0	CL

Missing values:

Bilirubin 0
 Edema 0
 Drug 106
 Stage 6
 Status 0

dtype: int64

Status value counts:

C 232
 D 161
 CL 25

Name: Status, dtype: int64

Edema value counts:

N 354
 S 44
 Y 20

Name: Edema, dtype: int64

(312, 5)

Documenting how I handled missing values.

Drug had 106 missing values and Stage had 6, while all other variables were complete. We dropped rows with missing values using `dropna()`, leaving a clean dataset.

#2

In [43]: `import statsmodels.api as sm`

```
df_c['is_dead'] = (df_c['Status'] == 'D').astype(int)
```

#dummy encode categorical variables

```
df_c_encoded = pd.get_dummies(df_c, columns=['Edema', 'Drug'], drop_first=True)
print(df_c_encoded.columns.tolist()) # check column names generated
```

```

features_c = ['Edema_S', 'Edema_Y', 'Drug_Placebo', 'Bilirubin']

X_c = sm.add_constant(df_c_encoded[features_c])
y_c = df_c['is_dead']

model_c = sm.Logit(y_c, X_c)
result_c = model_c.fit()

print(result_c.summary())

```

['Bilirubin', 'Stage', 'Status', 'is_dead', 'Edema_S', 'Edema_Y', 'Drug_Placebo']
 Optimization terminated successfully.

Current function value: 0.530596

Iterations 7

Logit Regression Results

```

=====
Dep. Variable:          is_dead    No. Observations:          312
Model:                Logit      Df Residuals:              307
Method:               MLE        Df Model:                  4
Date:                 Wed, 08 Apr 2026    Pseudo R-squ.:          0.2119
Time:                 03:02:15          Log-Likelihood:         -165.55
converged:            True          LL-Null:                -210.06
Covariance Type:      nonrobust        LLR p-value:            2.117e-18
=====

```

	coef	std err	z	P> z	[0.025	0.975]
const	-1.4087	0.235	-6.006	0.000	-1.868	-0.949
Edema_S	0.5783	0.461	1.254	0.210	-0.325	1.482
Edema_Y	2.8592	1.078	2.652	0.008	0.746	4.972
Drug_Placebo	-0.2527	0.269	-0.938	0.348	-0.781	0.275
Bilirubin	0.3504	0.064	5.443	0.000	0.224	0.477

```

=====

```

Does the drug improve survival probability? Does higher or lower bilirubin predict a higher or lower survival rate? How does survival rate vary with edema?

The drug (placebo coefficient: -0.253, $p=0.348$) does not have a statistically significant effect on survival, meaning D-penicillamine shows no meaningful improvement over placebo. Higher bilirubin strongly predicts higher mortality (coef --> +0.350 & $p<0.001$), so patients with more bilirubin in their blood are significantly more likely to die. Edema worsens survival substantially, having edema resolvable with diuretics (Edema_S, coef: +0.578) increases mortality risk, and having persistent edema despite diuretics (Edema_Y, coef: +2.859, $p=0.008$) dramatically increases it, making it the strongest predictor of death in the model.

#3

In [44]: `from sklearn.linear_model import LogisticRegression`

```

X_c_sk = df_c_encoded[features_c]
y_c_sk = df_c['is_dead']

```

```

model_c_sk = LogisticRegression(max_iter=1000)
model_c_sk.fit(X_c_sk, y_c_sk)

proba_c = model_c_sk.predict_proba(X_c_sk)

df_c_encoded['prob_dead'] = proba_c[:, 1]      # probability of death
df_c_encoded['prob_survival'] = proba_c[:, 0]    # probability of survival

print(df_c_encoded[['prob_dead', 'prob_survival']].describe())

```

	prob_dead	prob_survival
count	312.000000	312.000000
mean	0.400643	0.599357
std	0.238189	0.238189
min	0.181611	0.000170
25%	0.230736	0.510214
50%	0.291541	0.708459
75%	0.489786	0.769264
max	0.999830	0.818389

```

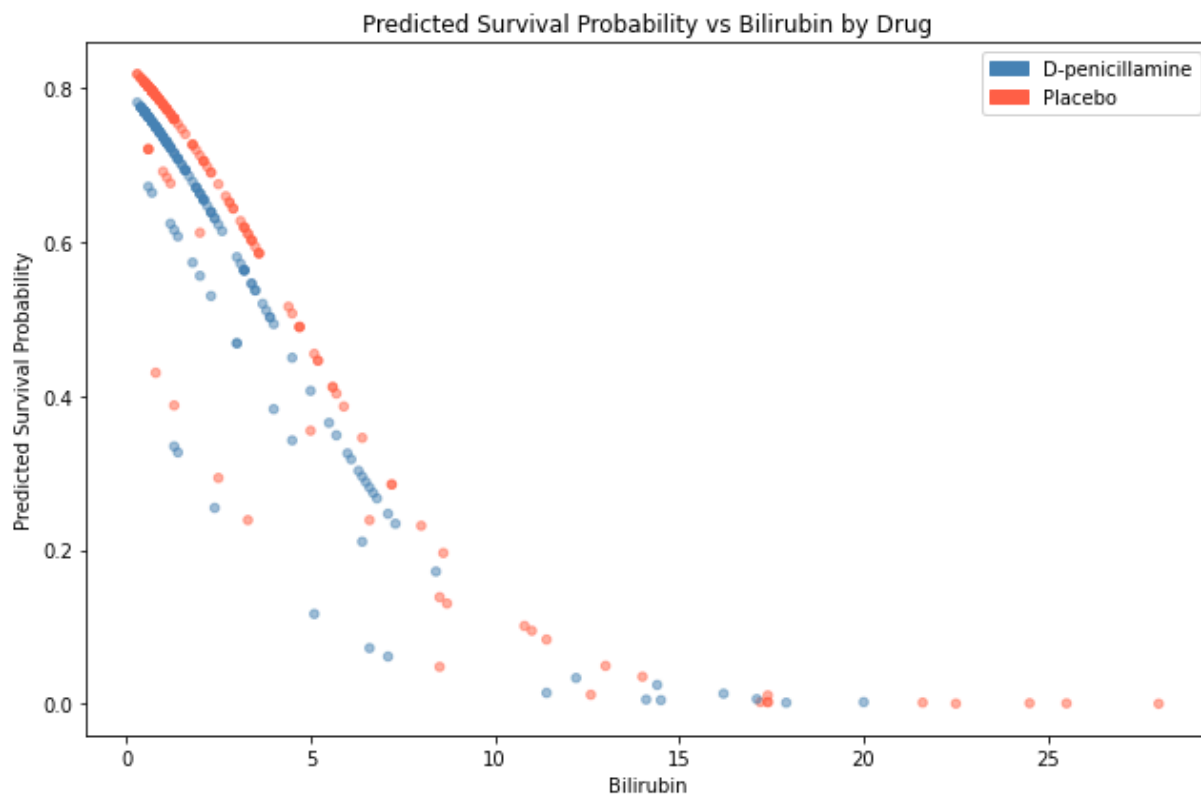
In [46]: colors = df_c_encoded['Drug_Placebo'].map({1: 'tomato', 0: 'steelblue'})

plt.figure(figsize=(9, 6))
plt.scatter(df_c['Bilirubin'], df_c_encoded['prob_survival'], c=colors, alpha=0.5,

import matplotlib.patches as mpatches
legend_handles = [
    mpatches.Patch(color='steelblue', label='D-penicillamine'),
    mpatches.Patch(color='tomato', label='Placebo')
]

plt.legend(handles=legend_handles)
plt.xlabel('Bilirubin')
plt.ylabel('Predicted Survival Probability')
plt.title('Predicted Survival Probability vs Bilirubin by Drug')
plt.tight_layout()
plt.show()

```



```
In [47]: mean_drug = df_c_encoded[df_c_encoded['Drug_Placebo'] == 0]['prob_survival'].mean()
mean_placebo = df_c_encoded[df_c_encoded['Drug_Placebo'] == 1]['prob_survival'].mean()
print(mean_drug - mean_placebo)
```

-0.018818509095048452

For what values of Bilirubin does the drug seem to increase patient survival rate? On average, by how much do patients who take the drug increase their probability of survival?

The drug appears to very slightly increase survival probability at low bilirubin levels (0-5), where the blue dots sit above the red dots, but the difference disappears at higher bilirubin levels. On average, taking D-penicillamine is associated with only a 1.9% point increase in survival probability, which is negligibly small and consistent with the regression showing the drug has no statistically significant effect.

#4

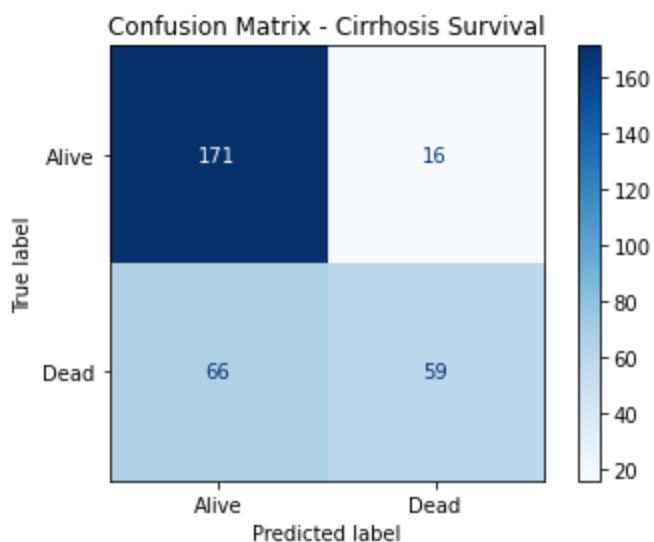
```
In [48]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

y_pred_c = model_c_sk.predict(X_c_sk)

cm_c = confusion_matrix(y_c_sk, y_pred_c, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm_c, display_labels=['Alive', 'Dead'])
disp.plot(cmap='Blues')
plt.title('Confusion Matrix - Cirrhosis Survival')
plt.show()
```



```
accuracy_c = (y_pred_c == y_c_sk).mean()
print(f"Accuracy: {accuracy_c:.4f}")
```



Accuracy: 0.7372

#5

```
In [49]: from sklearn.linear_model import LinearRegression
```

```
model_lpm_c = LinearRegression()
model_lpm_c.fit(X_c_sk, y_c_sk)
```

```
df_c_encoded['prob_survival_lpm'] = 1 - model_lpm_c.predict(X_c_sk)
```

```
print(df_c_encoded['prob_survival_lpm'].describe())
```

```
count    312.000000
mean      0.599359
std       0.227862
min      -0.441740
25%       0.582585
50%       0.684161
75%       0.719321
max       0.773868
Name: prob_survival_lpm, dtype: float64
```

```
In [50]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

```
axes[0].hist(df_c_encoded['prob_survival'], bins=40, color='steelblue', edgecolor='red')
axes[0].set_title('Logistic Regression')
axes[0].set_xlabel('Predicted Survival Probability')
axes[0].set_ylabel('Count')
axes[0].axvline(0, color='red', linestyle='--', linewidth=1)
axes[0].axvline(1, color='red', linestyle='--', linewidth=1)
```

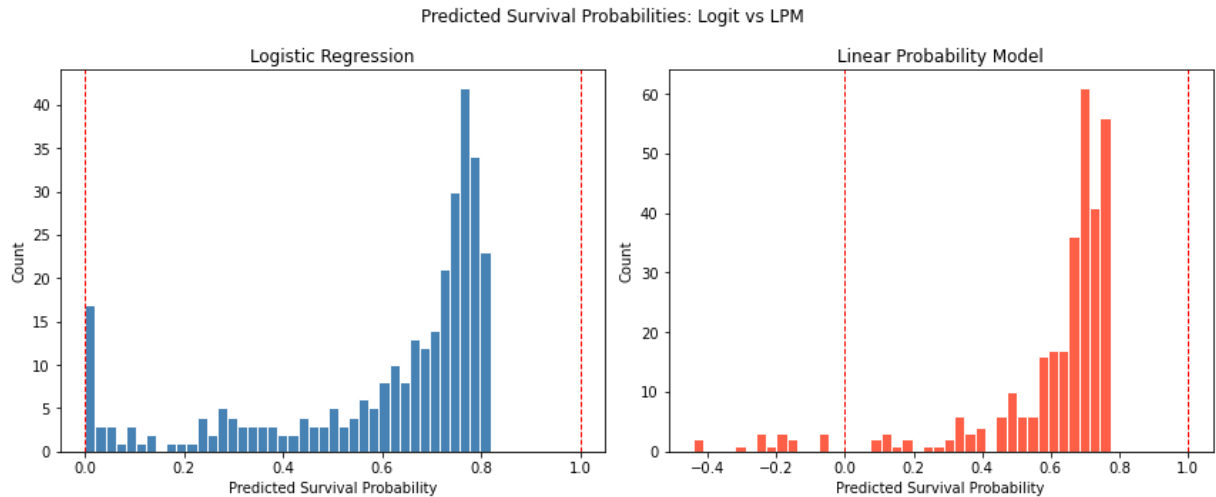
```
axes[1].hist(df_c_encoded['prob_survival_lpm'], bins=40, color='tomato', edgecolor='red')
axes[1].set_title('Linear Probability Model')
axes[1].set_xlabel('Predicted Survival Probability')
axes[1].set_ylabel('Count')
```

```

axes[1].axvline(0, color='red', linestyle='--', linewidth=1)
axes[1].axvline(1, color='red', linestyle='--', linewidth=1)

plt.suptitle('Predicted Survival Probabilities: Logit vs LPM')
plt.tight_layout()
plt.show()

```



#6

```

In [51]: from sklearn.linear_model import LogisticRegression

features_stage = ['Edema_S', 'Edema_Y', 'Bilirubin']

X_stage = df_c_encoded[features_stage]
y_stage = df_c['Stage'].astype(int)

model_stage = LogisticRegression(multi_class='multinomial', solver='lbfgs', max_iter=1000)
model_stage.fit(X_stage, y_stage)

print("Classes:", model_stage.classes_)
print("\nCoefficients:")
print(pd.DataFrame(model_stage.coef_, index=model_stage.classes_, columns=features_

```

Classes: [1 2 3 4]

Coefficients:

	Edema_S	Edema_Y	Bilirubin
1	-0.554367	-0.225352	-0.429724
2	-0.126235	-0.416191	0.076462
3	0.172828	-0.407497	0.152444
4	0.507775	1.049040	0.200819

```

In [52]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

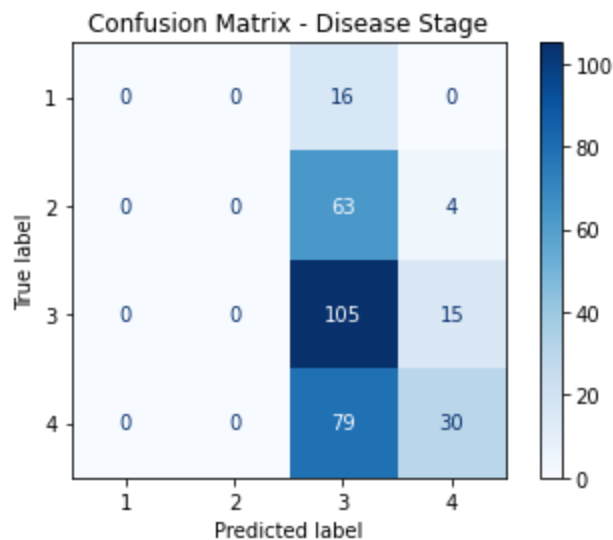
y_pred_stage = model_stage.predict(X_stage)

cm_stage = confusion_matrix(y_stage, y_pred_stage, labels=[1, 2, 3, 4])
disp = ConfusionMatrixDisplay(confusion_matrix=cm_stage, display_labels=[1, 2, 3, 4])
disp.plot(cmap='Blues')

```

```
plt.title('Confusion Matrix - Disease Stage')
plt.show()

accuracy_stage = (y_pred_stage == y_stage).mean()
print(f"Accuracy: {accuracy_stage:.4f}")
```



Accuracy: 0.4327

Describing my results above:

The model fails to predict Stages 1 and 2, classifying every patient into either Stage 3 or Stage 4, likely because Stages 1 and 2 are minority classes with too few patients to learn from. Stage 3 is the most common prediction and captures most true Stage 3 patients but also incorrectly absorbs nearly all Stage 1 and 2 patients. Overall accuracy is only 43.% and the model is basically acting as a binary classifier between Stages 3 and 4 rather than a proper 4-class predictor.

```
In [53]: proba_stage = model_stage.predict_proba(X_stage)

df_proba_stage = pd.DataFrame(proba_stage, columns=[f'Stage_{c}' for c in model_stage.classes_])
print(df_proba_stage.describe())
```

	Stage_1	Stage_2	Stage_3	Stage_4
count	3.120000e+02	312.000000	312.000000	312.000000
mean	5.129093e-02	0.214748	0.384613	0.349348
std	3.558217e-02	0.061014	0.061402	0.133760
min	2.988696e-09	0.012877	0.104297	0.255564
25%	1.804549e-02	0.192337	0.386800	0.270215
50%	5.441941e-02	0.246923	0.395665	0.292574
75%	8.495245e-02	0.253480	0.412569	0.373164
max	1.101252e-01	0.255116	0.428455	0.882826

Does your hard classification predict every class? Do your predicted probabilities predict every class?

The hard classification does not predict every class, it never predicts Stage 1 or Stage 2, only assigning patients to Stage 3 or Stage 4. The predicted

probabilities do cover all four stages, with every patient receiving a nonzero probability for each stage, but Stage 1 probabilities are very small (max is 0.11), reflecting the model's lower confidence in that class.

Q4. What are the coefficients of a logistic regression? How do you interpret their values?

1. For your linear model, your predictions are:

$$\hat{y} = b \cdot x = b_0 + \sum_{k=1}^K b_k x_k$$

What is the derivative of the prediction with respect to the k -th feature? For a 1 unit change in your feature, by how much does the prediction change?

2. For the logistic model, your predictions are:

$$\hat{p}_i = \frac{e^{b \cdot x}}{1 + e^{b \cdot x}}$$

What is the derivative of the prediction with respect to the k -th feature? As x changes, does your answer change? How is it similar or different from your answer for part 1?

Hint: It turns out that

$$\frac{\partial}{\partial L} \frac{e^L}{1 + e^L} = \frac{e^L}{(1 + e^L)^2} = \frac{e^L}{1 + e^L} \frac{1}{1 + e^L} = \frac{e^L}{1 + e^L} \left(1 - \frac{e^L}{1 + e^L} \right),$$

and use the chain rule ($D_x f(g(x)) = f'(g(x))g'(x)$). For what values of $\hat{p}$ is the coefficient itself (...times 4) roughly a good estimate of how a change in x_k affects the prediction?

3. Alternatively, we can transform the prediction like this:

$$\begin{aligned} \hat{p} &= \frac{e^{b \cdot x}}{1 + e^{b \cdot x}} \\ \hat{p}(1 + e^{b \cdot x}) &= e^{b \cdot x} \\ \hat{p} &= (1 - \hat{p})e^{b \cdot x} \\ \log\left(\frac{\hat{p}}{1 - \hat{p}}\right) &= b \cdot x \end{aligned}$$

The left-hand side of the above expression is called the **log odds ratio**, and the right-hand side is the latent variable. How does a one unit change in x_k affect the log odds ratio?

#1

The derivative of the prediction with respect to the "k-th" feature is just b_k (the coefficient for that feature). This means for every 1 unit increase in feature x_k , the predicted value $\hat{y}$

changes by exactly b_k . If b_k is positive the prediction goes up, if its negative it goes down. This is the key property of linear models, the effect of each feature is constant regardless of the values of the other features or the current value of x_k .

#2

The derivative of the predicted probability with respect to the "k-th" feature is $b_k * \hat{p} * (1 - \hat{p})$, where $\hat{p}$ is the current predicted probability. This comes from applying the chain rule to the logistic function. Unlike the linear model where the effect of x_k is always the constant b_k , here the effect changes depending on the current value of $\hat{p}$, meaning the same 1 unit change in x_k has a different impact on the predicted probability depending on where you are. The multiplier $\hat{p} * (1 - \hat{p})$ is maximized when $\hat{p} = 0.5$, where it equals 0.25, so the coefficient b_k times 0.25 is a good approximation of the marginal effect when the predicted probability is near 0.5. Far from $\hat{p} = 0.5$, near 0 or 1, the effect shrinks toward zero, meaning features matter less for predictions when the model is already very confident.

#3

The log odds ratio $\log(\hat{p} / (1 - \hat{p}))$, which equals b^*x , the linear combination of features and coefficients. A one unit increase in x_k increases the log odds ratio by exactly b_k , just like the linear model, this effect is constant and does not depend on the current value of $\hat{p}$. This is why logistic regression coefficients are interpreted as log odds ratios, rather than probability changes. Because on the log odds scale, the relationship is perfectly linear and constant, even though the effect on the actual probability ($\hat{p}$) varies depending on where you are.